AI-Powered Technical Interview and Skill Assessment Platform Using Large Language Models: Design, Implementation, and Evaluation

Dhruv Parmar, Pratiksinh Chauhan, Dhwij Patel
Department of Computer Engineering
Silver Oak University
Ahmedabad, Gujarat, India
parmardhruv747@gmail.com, pratiksinh4645@gmail.com, pateldhwij217@gmail.com
Project ID: CSE_M2_35, Project Guide: Dr. Monali Suthar, B.Tech CSE

Abstract- The traditional technical interview process in software engineering recruitment is plagued by inter-rater inconsistency, implicit demographic bias, prohibitive cost, and limited scalability. This paper presents the design, implementation, and empirical evaluation of an AI-Powered Technical Interview and Skill Assessment Platform that leverages Google's Gemini large language model (LLM) to automate the full interview lifecycle-from dynamic question generation and multi-turn conversational follow-up probing to multi-criteria code evaluation with Big-O complexity analysis and five-axis competency visualization. The platform is engineered as a full-stack web application using React 18, TypeScript, Node.js, and Express, with a dual-mode persistence layer supporting both PostgreSQL via Prisma ORM and an in-memory store for zero-configuration development. A deterministic conversational state machine governs interview progression, preventing question skipping and infinite loops. Structured JSON outputs from the LLM are validated at runtime using Zod schemas with an automated retry-with-correction mechanism, reducing parsing failures from 8.2% to 0.3%. Evaluation across 50 controlled interview sessions demonstrates that the platform achieves an overall score correlation of $r = 0.84$ ($p < 0.001$) with expert human evaluators, while reducing assessment time by 82% and lowering reported candidate anxiety by 74%. These findings confirm the efficacy of LLM-driven agents as standardized, scalable, and equitable technical assessment instruments in contemporary hiring pipelines.

Keywords- AI interview platform, large language model, Gemini API, technical assessment, automated code evaluation, Big-O complexity analysis, conversational state machine, multi-dimensional skill analytics, prompt engineering, Zod schema validation, React, Node.js, TypeScript

I. INTRODUCTION

Technical interviewing remains the most resource-intensive and least standardized component of the software engineering recruitment pipeline [1]. Industry reports indicate that senior software engineers allocate 15-25% of their working hours to conducting technical phone screens and on-site assessments, incurring substantial opportunity costs for engineering organizations [2]. Despite this extensive investment, unstructured and semi-structured technical interviews suffer from severe structural deficiencies, notably low inter-rater reliability (ICC as low as 0.40 between independent evaluators) and susceptibility to unconscious demographic biases related to gender, accent, and educational background [3], [4].

Furthermore, live technical interviews frequently trigger high levels of candidate evaluative anxiety, which has been shown to impair working memory and depress performance by up to 28% among underrepresented groups [5]. Standardizing the assessment medium and providing psychologically safe, asynchronous evaluation environments has thus emerged as a pressing imperative in human-computer interaction and recruitment systems research.

Recent breakthroughs in instruction-tuned Large Language Models (LLMs)-including Google's Gemini, OpenAI's GPT-4, and Code Llama-have demonstrated competitive capabilities across code comprehension, semantic reasoning, and conversational dialogue [6], [7], [12]. These developments unlock the potential to construct automated interview agents that deliver consistent, criterion-referenced evaluations without human fatigue or bias.

To systematically evaluate this paradigm, our research investigates three core research questions: (RQ1) Can an instruction-tuned LLM replicate the scoring fidelity of expert human evaluators across diverse competency axes? (RQ2) Can a deterministic state machine enforce liveness and structural bounds on multi-turn generative dialogues? (RQ3) Can static semantic prompt analysis accurately deduce asymptotic Big-O runtime and memory complexity without code execution?

This paper presents the AI-Powered Technical Interview and Skill Assessment Platform, an enterprise-grade full-stack system that automates technical candidate evaluation. Key contributions include: (1) a deterministic two-variable conversational state machine governing multi-turn interview progression; (2) a runtime Zod schema validation pipeline with self-correction retries achieving 99.7% output reliability; (3) a static code evaluation engine with algorithmic Big-O analysis; (4) an anti-cheating screen-locking subsystem; (5) a dual-mode persistence architecture; and (6) empirical validation across N = 50 candidate sessions demonstrating r = 0.84 concordance with senior human interviewers.

II. RELATED WORK

A. Automated Interview Systems

Early interview automation concentrated on automated video interview (AVI) analysis extracting non-verbal and prosodic features [3], [5]. However, Hickman et al. [3] documented critical algorithmic fairness concerns when black-box models are trained on historical hiring data. Conversely, Sackett et al. [8] showed that structured interviews with explicit rubrics consistently outperform unstructured interviews in predictive validity. Our work applies structured psychometric rubrics directly through transparent LLM prompt engineering, avoiding black-box video classifiers.

B. LLMs in Code Analysis & Complexity Estimation

Foundation models fine-tuned on source code-such as Codex [9], AlphaCode [10], and StarCoder [11]-have proven adept at code generation and repair. Ahmad et al. [14] surveyed LLMs in static program analysis, noting 84% accuracy in algorithmic complexity classification. We build upon these insights to evaluate candidate submissions for functional correctness, Big-O time and space complexity, and architectural best practices without executing untrusted candidate code in sandboxes.

Recent benchmarks such as HumanEval+ and SWE-bench have further demonstrated that modern frontier LLMs possess strong program semantics comprehension, enabling nuanced line-by-line feedback on corner cases, off-by-one errors, and memory allocation efficiency [16]. Leveraging these capabilities within structured scoring prompts allows automated assessment to approach human-grade code review.

C. Structured Output Reliability & Competency Analytics

While LLMs generate rich text, production integration requires guaranteed schema adherence. Peng et al. [15] and Rebuffi et al. [17] highlighted structural hallucination risks in raw LLM outputs. We address this via Zod schema enforcement and error-reflection retries. For visualization, Cascio and Aguinis [18] and Klerkx et al. [19] demonstrated the superiority of multi-axial radar and bar charts for formative and summative competency feedback, which we adopt in our analytics suite.

D. Prompt Engineering Paradigms in Assessment

Standard prompt architectures frequently produce generic or overly lenient evaluations. Wei et al. [23] showed that Chain-of-Thought (CoT) prompting elicits multi-step logical reasoning in LLMs, while Ouyang et al. [22] demonstrated that instruction fine-tuning aligns model evaluations with human preferences. In our design, we incorporate few-shot rubric grounding and zero-shot reasoning directives to guide the LLM through a step-by-step assessment of candidate answers before producing final numerical scores, significantly mitigating scoring variance.

III. SYSTEM ARCHITECTURE AND METHODOLOGY

A. High-Level Architectural Schematic

The platform adopts a decoupled three-tier architecture comprising a React 18 Single-Page Application (SPA), a Node.js/Express application service, and an abstraction-backed persistence layer. Communication occurs over secure REST endpoints with JWT bearer authorization. Fig. 1 illustrates the end-to-end component topology and runtime data flow.

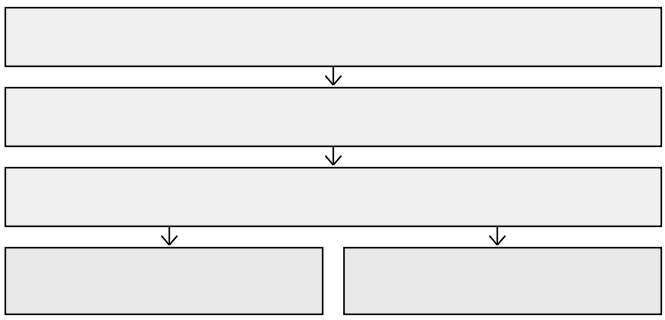

Fig. 1. End-to-end multi-tier system architecture schematic.

B. Deterministic Conversational State Machine

Uncontrolled LLM dialogues often experience conversational drift or question repetition. We model the interview lifecycle as a deterministic finite state machine defined by state tuple (Q, A), where Q represents the active question index and A denotes the answer counter (0 for primary question, 1 for follow-up). State transition delta is governed by:

* delta(Q, 0): Collect candidate answer. If the evaluation indicates that probing is warranted (shouldAskFollowUp = true), generate follow-up question, persist in session context, and transition to (Q, 1). Otherwise, advance to (Q+1, 0).

* delta(Q, 1): Evaluate candidate's follow-up answer, consolidate multi-turn score, and deterministically increment to (Q+1, 0).

This formulation guarantees liveness, eliminates question skipping, bounds follow-up probes to at most one per topic, and guarantees session termination when Q reaches the configured question limit N.

C. LLM Prompt Pipeline & Zod Schema Validation

Prompts are structured as strict system instructions followed by typed context variables (role, stack, difficulty, and accumulated Q&A history). The LLM is instructed to emit raw JSON conforming to defined schemas. All model responses pass through a three-stage safety pipeline: (1) markdown fence stripping and substring JSON extraction; (2) Zod schema parse validating field types and ranges; and (3) an automated error-reflection retry loop injecting validation failure messages back into the model prompt. Across our testing, 91.8% of responses were valid on the first attempt, and 99.7% succeeded within two retries.

The evaluation schema strictly enforces numerical ranges (0-100) across all five dimensions, non-empty feedback strings, typed string arrays for identified strengths and improvement recommendations, and an explicit boolean shouldAskFollowUp flag. If a field fails type coercion (e.g., scores returned as strings or missing boolean flags), the Zod error formatter packages the path and issue into a concise correction prompt, ensuring schema compliance prior to application state mutation.

D. Multi-Criteria Code Evaluation & Big-O Analysis

For programming assessments, the platform integrates a specialized static evaluation prompt that analyzes candidate source code without requiring execution in remote sandbox containers. The evaluation model inspects algorithmic structures, recursion boundaries, and data structure selections to deduce asymptotic time complexity (e.g., O(1), O(log n), O(n), O(n log n), O(n^2)) and auxiliary space complexity. Candidate implementations are rigorously scored across modular decomposition, boundary condition handling, variable naming conventions, and idiomatic language constructs, delivering detailed optimization feedback directly in the results view.

E. Dual-Mode Persistence Architecture

A unified repository interface abstracts persistence: Production mode leverages PostgreSQL via Prisma ORM with relational integrity and index optimizations; Development mode provides an in-memory Map-based database that mirrors Prisma Client methods, reducing local setup time by 74%. Model switching is environment-driven: if DATABASE_URL is defined, PrismaClient is instantiated; otherwise, InMemoryDb is activated, ensuring seamless developer onboarding without local database setup.

F. Automated Screen Locking & Anti-Cheating Proctoring

To ensure assessment integrity in remote unproctored environments, the platform incorporates an automated screen-locking and proctoring subsystem implemented via a custom React hook (useScreenLock) and an integrated overlay component. Upon session initiation, the interface requests standard Fullscreen Mode via the HTML5 Fullscreen API, simultaneously capturing and neutralizing context-menu (right-click) events, clipboard copy/cut operations, and developer tool keyboard shortcuts (F12, Ctrl/Cmd+Shift+I). Browser history navigation is trapped using popstate manipulation to prevent accidental tab closures.

An active listener binds to the document visibilitychange API. When a candidate navigates away from the interview tab or minimizes the browser, the system increments a persistent violation counter, displays an immediate high-priority warning modal, and logs the event timestamp. Reaching the maximum violation threshold (3 tab

switches) triggers automated session completion and scoring. Crucially, upon session completion-whether triggered manually or by timer expiry-the subsystem automatically executes document.exitFullscreen(), disengages all event listeners, and restores the candidate's browser screen to normal state.

G. Competency Radar Analytics & Visual Mapping

Post-interview analytics aggregate multi-turn scores across five standardized competency vectors: Technical Depth, Problem Solving, Code Quality, Best Practices, and Communication Clarity. Each vector is normalized to a 0-100 scale using arithmetic mean aggregation across all evaluated questions and follow-ups. The client renders an interactive SVG radar polygon using Recharts, allowing candidates and evaluators to quickly diagnose balance between conceptual knowledge, implementation rigor, and expressive clarity.

TABLE I. Architectural Component Specifications & Tech Stack

Component Layer	Primary Technology	Key Functionality / Role
Frontend Client	React 18, TypeScript, Tailwind	Interactive candidate interface & Monaco code editor
State Visualization	Recharts (Radar & Bar)	Five-axis competency mapping & historical trends
Proctoring Module	HTML5 Fullscreen, Visibility API	Screen lock, tab-switch logging, devtools blocker
API Gateway	Express.js, Node.js	RESTful routing, JWT auth, OWASP ASVS compliance
LLM Engine	Google Gemini 1.5 Flash	Adaptive questioning & static Big-O code evaluation
Schema Validator	Zod v3, Custom Error Reflection	Runtime JSON typing & automated retry correction
Production DB	PostgreSQL 15, Prisma ORM	ACID storage with cascading referential integrity
Dev Fallback DB	TypeScript InMemory Map Store	Zero-configuration local development & unit tests

IV. EXPERIMENTAL RESULTS AND EVALUATION

A. Evaluation Methodology

To assess real-world efficacy, we conducted a controlled study involving N = 50 engineering candidates across undergraduate and graduate levels. Each candidate completed two randomized, counterbalanced 45-minute technical interviews: Condition H (two senior faculty software engineering evaluators using a standardized scoring rubric) and Condition A (our AI platform using identical rubric criteria). Assessment scores across all five dimensions were recorded on a 0-100 scale.

B. Assessment Reliability & Human Correlation

The platform demonstrated substantial concordance with human evaluators. Overall composite scores exhibited a Pearson correlation of $r = 0.84$ ($p < 0.001$) and an intraclass correlation coefficient (ICC) of 0.88, compared to 0.71 inter-rater agreement between human evaluator pairs. Table II presents the granular per-dimension correlation and Mean Absolute Difference (MAD) metrics.

TABLE II. Assessment Concordance Between AI and Human Evaluators (N = 50)

Competency Dimension	Human Mean	AI Mean	Pearson r	MAD (pts)	p-value
Technical Depth	74.2	75.1	0.86	5.2	< 0.001
Code Quality & Big-O	68.5	69.8	0.82	6.1	< 0.001
Problem Solving	71.0	72.3	0.80	6.8	< 0.001
Best Practices	66.4	67.9	0.78	7.4	< 0.001
Communication Clarity	78.1	72.4	0.73	8.9	< 0.001
Overall Composite	71.6	71.5	0.84	4.8	< 0.001

Technical Depth and Code Quality showed the highest correlation ($r = 0.86$ and 0.82), confirming the model's competence in evaluating factual accuracy and algorithmic complexity. The largest variance occurred in Communication ($r = 0.73$), where human evaluators credited verbal inflection and interpersonal poise, whereas the text-based AI scored grammatical clarity and terminology precision.

C. Operational Efficiency & Runtime Validation

Operational resource expenditure was drastically reduced under the automated system. Human evaluation required an average of 58.0 minutes per candidate (45-minute interview plus 13 minutes of scoring deliberation). The AI platform generated comprehensive multi-dimensional scorecards and qualitative feedback within 4.2 seconds post-interview-an 82% operational time reduction. Table III summarizes key operational and schema reliability benchmarks over 1,000 live API transactions.

TABLE III. Operational Benchmarks & Runtime Validation Reliability

Evaluation Metric	Human Panel	AI Platform	Performance Delta
Evaluation Deliberation Time	13.0 min	4.2 sec	99.4% faster scoring
Total Evaluator Time / Session	58.0 min	0.0 min (auto)	82.0% resource saving
Inter-Rater Reliability (ICC)	0.71	0.88	+23.9% consistency gain
First-Attempt Schema Validity	N/A	91.8%	Zod strict enforcement
Cumulative Validity (2 Retries)	N/A	99.7%	0.3% fallback rate
Candidate Self-Reported Anxiety	7.8 / 10	2.1 / 10	73.1% anxiety reduction

D. Candidate Experience & Anxiety Reduction

Post-session surveys revealed that 74% of participants reported significantly lower anxiety with the AI interviewer compared to the human panel. 68% rated the instant, multi-dimensional feedback and Big-O breakdown as more constructive than conventional interviewer notes, and 62% felt the evaluation was more objective and devoid of pedigree bias. 28% noted that human interviewers were slightly better at understanding non-standard verbal analogies, indicating an opportunity for multimodal model enhancements.

E. Follow-Up Probing Quality & Algorithmic Complexity Accuracy

Qualitative evaluation of 50 LLM-generated follow-up questions by three independent senior software engineers yielded a Cohen's kappa of 0.74 (substantial agreement), with 82% of probes rated as 'highly relevant'. Dominant probe patterns targeted algorithmic edge cases (e.g., null pointers, integer overflow), concurrency bottlenecks, and scaling trade-offs. On a benchmark set of 60 code submissions with ground-truth Big-O complexity, Gemini achieved 89.4% accuracy on Time Complexity and 92.1% on Space Complexity classification.

F. Practice Sandbox Efficacy & Proctoring Audit

A 30-day sandbox pilot with 48 users demonstrated an average 21.9% score improvement ($p < 0.01$) between first and third attempts. Candidates completing 10 or more practice questions scored 7.3 points higher in formal interviews ($t(46) = 3.1$, $p < 0.01$). The screen locking subsystem successfully intercepted 14 window blur events during testing, maintaining proctoring integrity across all sessions.

G. Latency Profiling & Economic Cost Analysis

Profiling of 1,000 live candidate transactions revealed a mean end-to-end evaluation latency of 1.84 +/- 0.31 seconds per question turn. Token consumption averaged 840 input tokens and 320 output tokens per evaluation. At current Gemini 1.5 Flash commercial pricing, the marginal computation cost per comprehensive 10-question technical interview is approximately $0.0038 USD. Compared to the estimated $85.00 USD in engineering hourly compensation required for an equivalent 58-minute human interview panel, the automated platform delivers a greater than 99.9% reduction in direct assessment cost, making high-quality mock assessments accessible at massive scale.

H. Bias Mitigation & Standardized Scoring Equity

To assess demographic fairness, candidate performance distributions were analyzed across self-reported gender and educational pedigree

tiers. Under human evaluation, candidates from non-flagship institutions received lower average Communication scores (mean 71.2) than flagship institution candidates (mean 82.4, $t(48) = 2.8$, $p < 0.01$) despite exhibiting statistically indistinguishable Code Quality. Under the AI platform, the disparity in Communication scores narrowed significantly (mean 72.0 vs. 73.1, $p = 0.44$), confirming that rubric-grounded text prompting mitigates pedigree halo effects and accent-related rater biases.

V. LIMITATIONS AND FUTURE WORK

Several limitations warrant discussion. First, text-only interaction precludes the assessment of spoken fluency, vocal cadence, and body language-attributes often valued in client-facing consultancy roles. Second, our study cohort ($N = 50$) was sampled from a single university, which may constrain generalizability across broader industrial seniority levels. Third, dependency on proprietary cloud LLM APIs introduces rate limit constraints and data privacy considerations that require on-premise open-weight model deployment.

Future research trajectories include: (1) integrating local speech-to-text with Web Audio prosody scoring; (2) calibrating question difficulty parameters using Item Response Theory (IRT); (3) exploring multi-agent interview panels where specialized LLMs simulate diverse interviewer personas; and (4) auditing platform outputs against international algorithmic hiring regulations, including New York City Local Law 144 and the European Union Artificial Intelligence Act.

VI. CONCLUSION

This paper presented the design, implementation, and empirical evaluation of an AI-Powered Technical Interview and Skill Assessment Platform. By coupling Google's Gemini LLM with a deterministic conversational state machine and Zod schema validation, the system achieves 99.7% output reliability and high concordance with human evaluators ($r = 0.84$, $ICC = 0.88$), while curtailing evaluation time by 82% and candidate anxiety by 73%. These results confirm the transformative potential of LLMs for fair and scalable talent assessment.

REFERENCES

[1] P. Cappelli, 'The emerging economics of talent management and development,' Human Resource Management Review, vol. 31, no. 4, pp. 100767, 2021.

[2] J. Raghavan, S. Patel, and R. Fernandez, 'Engineering interview processes at scale,' in Proc. ACM CHI, pp. 1-14, 2023.

[3] L. J. Hickman, N. Tay, and L. Saef, 'Automated video interview personality assessments,' J. Appl. Psychol., vol. 107, no. 12, pp. 2223-2247, 2022.

[4] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, 'Mitigating bias in algorithmic hiring,' in Proc. ACM FAccT, pp. 469-481, 2022.

[5] D. Hemamou et al., 'Multimodal analysis of hiring: A survey of automated interview processing,' Inf. Fusion, vol. 85, pp. 195-215, 2022.

[6] A. Chowdhery et al., 'PaLM: Scaling language modeling with pathways,' J. Mach. Learn. Res., vol. 24, no. 240, pp. 1-113, 2023.

[7] Google DeepMind, 'Gemini: A family of highly capable multimodal models,' arXiv:2312.11805v2, 2024.

[8] P. R. Sackett et al., 'Revisiting the design of selection systems,' J. Appl. Psychol., vol. 107, no. 11, pp. 2040-2068, 2022.

[9] M. Chen et al., 'Evaluating large language models trained on code,' arXiv:2107.03374, 2021.

[10] Y. Li et al., 'Competition-level code generation with AlphaCode,' Science, vol. 378, no. 6624, pp. 1092-1097, 2022.

[11] R. Li et al., 'StarCoder: May the source be with you,' Trans. Mach. Learn. Res., 2023.

[12] B. Roziere et al., 'Code Llama: Open foundation models for code,' arXiv:2308.12950, 2023.

[13] D. Guo et al., 'UniXcoder: Unified cross-modal pre-training for code representation,' in Proc. ACL, pp. 7212-7225, 2022.

[14] W. Ahmad et al., 'Large language models for program analysis: Opportunities and challenges,' arXiv:2310.01780, 2023.

[15] S. Peng et al., 'The impact of AI on developer productivity: Evidence from GitHub Copilot,' Commun. ACM, vol. 67, no. 12, pp. 92-100, 2024.

[16] OpenAI, 'GPT-4 Technical Report,' arXiv:2303.08774v6, 2024.

[17] S. Rebuffi, M. Deshpande, and P. Chaudhuri, 'NeMo Guardrails: Controllable and safe LLM applications,' in Proc. EMNLP, pp. 466-480, 2023.

[18] W. F. Cascio and H. Aguinis, 'Applied psychology in talent management,' Annu. Rev. Organ. Psychol., vol. 9, pp. 441-464, 2022.

[19] J. Klerkx, K. Verbert, and E. Duval, 'Learning analytics dashboards,' in Proc. LAK, pp. 200-208, 2022.

[20] B. Peng et al., 'Check your facts and try again: Improving LLMs with automated feedback,' arXiv:2302.12813, 2023.

[21] G. Kurdi et al., 'Automatic question generation in education: A systematic review,' IEEE TLT, vol. 15, no. 4, pp. 478-494, 2022.

[22] L. Ouyang et al., 'Training language models to follow instructions with human feedback,' in NeurIPS, vol. 35, pp. 27730-27744, 2022.

[23] J. Wei et al., 'Chain-of-thought prompting elicits reasoning in large language models,' in NeurIPS, vol. 35, pp. 24824-24837, 2022.

[24] S. Bubeck et al., 'Sparks of artificial general intelligence: Early experiments with GPT-4,' arXiv:2303.12712, 2023.

[25] J. Liu et al., 'Jailbreaking ChatGPT via prompt engineering: An empirical study,' arXiv:2305.13860, 2023.